

Moteur de jeu Sokoban — Guide d'utilisation

Ce document décrit le fonctionnement du moteur de jeu Sokoban fourni. Vous devez utiliser ces classes **sans les modifier** pour développer votre interface graphique JavaFX.

Structure du moteur

Package `ihm.sokoban.model`

Classe	Description
<code>JeuSokoban</code>	Classe principale : grille, déplacement, poussée, undo, gestion des niveaux, détection de blocage
<code>Direction</code>	Enum : <code>HAUT</code> , <code>BAS</code> , <code>GAUCHE</code> , <code>DROITE</code> (avec décalages dl/dc)
<code>TypeCase</code>	Enum : <code>MUR</code> , <code>SOL</code> , <code>CIBLE</code> , <code>CAISSE</code> , <code>CAISSE_SUR_CIBLE</code> , <code>JOUEUR</code> , <code>JOUEUR_SUR_CIBLE</code> , <code>VIDE</code>
<code>EtatPartie</code>	Enum : <code>EN_COURS</code> , <code>GAGNEE</code> , <code>PERDU</code>
<code>ResultatMouvement</code>	Enum : <code>DEPLACE</code> , <code>POUSSE</code> , <code>BLOQUE</code> (<code>NIVEAU_TERMINE</code> n'est plus retourné — voir exception)
<code>SokobanException</code>	Exception en cas d'erreur (niveau invalide, rien à annuler, mouvement après fin , etc.)

Package `ihm.sokoban.util`

Classe	Description
<code>NiveauxTutoriel</code>	Banque de 5 niveaux 8×8 pour le palier A (mode tutoriel)
<code>NiveauxSokoban</code>	Banque de 10 niveaux variés pour le palier B
<code>LoaderNiveauxXSB</code>	Chargeur de dossiers <code>.xsb</code> pour le palier C

Utilisation de `JeuSokoban`

Créer une partie

Trois constructeurs sont disponibles selon le **palier** visé :

```
// --- Palier A (mode tutoriel : 5 niveaux 8×8) ---
JeuSokoban jeu = new JeuSokoban(
    NiveauxTutoriel.getNiveaux(),
    NiveauxTutoriel.getNoms(),
    0    // index du niveau initial
```

```

);

// --- Palier B (10 niveaux standards) ---
JeuSokoban jeu = new JeuSokoban(0); // équivalent à NiveauxSokoban

// --- Palier C (dossier .xsb) ---
import ihm.sokoban.util.LoaderNiveauxXSB;
import ihm.sokoban.util.LoaderNiveauxXSB.Banque;
import java.nio.file.Path;

Banque b =
LoaderNiveauxXSB.chargerDepuisDossier(Path.of("niveaux_xsb_exemple"));
JeuSokoban jeu = new JeuSokoban(b.niveaux, b.noms, 0);

// --- Cas particulier : un seul niveau, sans banque ---
String niveau = "#####\n#      #\n#  @ $ #\n#      . #\n#      #\n#####";
JeuSokoban jeu = new JeuSokoban(niveau);

```

Changer de banque à la volée

Pour basculer entre paliers depuis un menu :

```

jeu.setBanqueNiveaux(NiveauxTutoriel.getNiveaux(),
NiveauxTutoriel.getNoms());
// ou
jeu.setBanqueNiveaux(NiveauxSokoban.getNiveaux(),
NiveauxSokoban.getNoms());
// ou
jeu.setBanqueNiveaux(b.niveaux, b.noms);

```

Déplacer le joueur

```

// IMPORTANT : vérifier l'état AVANT d'appeler deplacer().
// Le moteur lève une SokobanException (MOUVEMENT_APRES_FIN) si on
// l'appelle alors que la partie est gagnée ou perdue.
if (jeu.peutJouer()) {
    ResultatMouvement r = jeu.deplacer(Direction.DROITE);
    switch (r) {
        case DEPLACE: /* Le joueur s'est déplacé (sans pousser) */ break;
        case POUSSE:  /* Le joueur a poussé une caisse */ break;
        case BLOQUE:  /* Mouvement impossible (mur ou caisse non poussable)
*/ break;
        default: break;
    }
}

```

Lire l'état du jeu

```
// État de la partie
EtatPartie etat = jeu.getEtatPartie(); // EN_COURS, GAGNEE ou PERDU
boolean enCours = jeu.peutJouer(); // raccourci : etat == EN_COURS
boolean termine = jeu.isNiveauTermine(); // raccourci : etat == GAGNEE
boolean perdu = jeu.isPerdu(); // raccourci : etat == PERDU
boolean dernier = jeu.estDernierNiveau(); // utile pour message différencié

// Position du joueur
int ligneJoueur = jeu.getJoueurLigne();
int colonneJoueur = jeu.getJoueurColonne();

// Compteurs
int mouvements = jeu.getNbMouvements();
int poussees = jeu.getNbPoussees();
int caissesPlacees = jeu.getNbCaissesSurCible();
int caissesTotal = jeu.getNbCaisses();

// Dimensions de la grille
int lignes = jeu.getNbLignes();
int colonnes = jeu.getNbColonnes();

// Contenu d'une case
TypeCase tc = jeu.getCas(e(ligne, colonne);

// Copie complète de la grille
TypeCase[][] grille = jeu.getGrille();
```

Exploiter TypeCase

L'enum `TypeCase` offre des méthodes utilitaires très pratiques :

```
TypeCase tc = jeu.getCas(e(ligne, colonne);

tc.estMur() // true si c'est un mur
tc.estCaisse() // true si c'est une caisse (avec ou sans cible)
tc.estCible() // true si c'est une cible (vide, avec caisse, ou avec
joueur)
tc.estJoueur() // true si le joueur est ici
tc.estLibre() // true si sol ou cible vide (on peut y marcher)
```

Annuler un mouvement (Undo)

```
if (jeu.peutAnnuler()) {
    jeu.annuler();
    // La grille revient à l'état précédent
}
```

Recommencer le niveau

```
jeu.reset(); // Recharge le niveau courant depuis le début
```

Naviguer entre les niveaux

```
// Niveau courant
int index = jeu.getNiveauCourant();
int total = jeu.getNbNiveaux();
String nom = jeu.getNomNiveauCourant(); // ex : "Pousser une caisse"
boolean estDernier = jeu.estDernierNiveau();

// Passer au suivant / précédent
boolean ok = jeu.niveauSuivant(); // false si c'est le dernier
boolean ok2 = jeu.niveauPrecedent(); // false si c'est le premier

// Charger un niveau spécifique
jeu.chargerNiveauParIndex(5);
```

Informations sur les niveaux

```
import ihm.sokoban.util.NiveauxSokoban;
import ihm.sokoban.util.NiveauxTutoriel;

// Banque standard (palier B) : 10 niveaux
int nb = NiveauxSokoban.getNbNiveaux(); // 10
String nom = NiveauxSokoban.getNomNiveau(0); // "Tutoriel"
String[] niveaux = NiveauxSokoban.getNiveaux();
String[] noms = NiveauxSokoban.getNoms();

// Banque tutoriel (palier A) : 5 niveaux 8x8
int nbTuto = NiveauxTutoriel.getNbNiveaux(); // 5
int taille = NiveauxTutoriel.TAILLE; // 8
String nomTuto = NiveauxTutoriel.getNomNiveau(0); // "Pousser une
caisse"
```

Charger un dossier .xsb (palier C)

```
import ihm.sokoban.util.LoaderNiveauxXSB;
import ihm.sokoban.util.LoaderNiveauxXSB.Banque;
import java.nio.file.Path;

Banque b =
LoaderNiveauxXSB.chargerDepuisDossier(Path.of("niveaux_xsb_exemple"));
```

```
System.out.println(b.niveaux.length + " niveaux chargés.");
// Lève SokobanException si le dossier est invalide ou vide.
```

Affichage console (debug)

```
System.out.println(jeu);
```

Gestion des erreurs

TypeErreur	Cause
NIVEAU_INVALIDE	La chaîne de niveau est null/vide, ou le dossier <code>.xsb</code> est invalide
AUCUN_JOUEUR	Le niveau ne contient pas de '@' ou '+'
PLUSIEURS_JOUEURS	Le niveau contient plus d'un joueur
CAISSES_CIBLES_DIFFERENTES	Le nombre de caisses $\neq$ nombre de cibles
NIVEAU_INEXISTANT	L'index de niveau est hors limites
RIEN_A_ANNULER	Tentative d'annuler sans historique
MOUVEMENT_APRES_FIN	Tentative de <code>deplacer()</code> alors que la partie est gagnée ou perdue

Toutes ces erreurs lancent une `SokobanException` (`RuntimeException`). C'est à votre IHM de **filtrer en amont** :

- Vérifier `jeu.peutJouer()` avant d'appeler `deplacer()`.
- Vérifier `jeu.peutAnnuler()` avant d'appeler `annuler()`.
- Vérifier que le dossier existe avant `chargerDepuisDossier()`.

Si une exception remonte malgré tout, attrapez-la pour afficher un message convivial (et pas planter le programme).

Conseils pour l'implémentation JavaFX

Composants utiles

- **GridPane** pour la grille du plateau
- **Label** ou **Rectangle** pour chaque case (avec classe CSS par type)
- **Label** pour les compteurs et messages
- **MenuBar** pour les menus (Jeu / Mode / Édition / Aide)
- **Alert** pour les dialogues (victoire, défaite, confirmation)
- **DirectoryChooser** pour le palier C (sélection d'un dossier `.xsb`)

Déplacements et gestion du clavier

Vous pouvez proposer des boutons pour les déplacements, mais Le Sokoban se joue souvent au clavier.

⚠ Piège classique : si vous utilisez `scene.setOnKeyPressed(...)`, dès qu'un `Button` ou la `MenuBar` aura le focus (après un clic souris ou la fermeture d'une `Alert`), les flèches **arrêteront de fonctionner** : elles seront mangées par le focus traversal de JavaFX.

Bonne approche : utiliser un `EventFilter` sur la `Scene` (phase de capture) et **consommer** l'événement.

```
import javafx.scene.input.KeyEvent;

scene.addEventFilter(KeyEvent.KEY_PRESSED, event -> {
    boolean traite = true;
    switch (event.getCode()) {
        case UP:      /*...*/ break;
        case DOWN:    /*...*/ break;
        /*...*/
        default: traite = false;
    }
    if (traite) event.consume();
});
```

Le mini projet `gestion_clavier.zip` (sur Moodle) illustre concrètement la différence entre la mauvaise et la bonne approche. **À lancer avant de coder votre IHM.**

Gérer les fins de partie (gagné / perdu / dernier niveau)

Vous devez vérifier l'état de la partie après chaque mouvement pour afficher un message adapté.

Après GAGNEE ou PERDU, le joueur ne peut plus se déplacer (le moteur lève une exception).

Seuls `annuler()`, `reset()` et `niveauSuivant()` restent utiles.